

Hual: A RAW Photo Ingestion and Cataloguing Engine

Hari Narendranath

July 29, 2026

Contents

1	Overview	2
2	Thumbnail Extraction	2
2.1	Zero-Copy mmap Parsing	2
2.2	TIFF/EXIF Parser (NEF, ARW, DNG, plain JPEG EXIF)	2
2.3	ISO-BMFF Box Parser (Canon CR3)	3
2.4	JPEG Parser	3
2.5	Convergence Design	3
3	Caching Architecture	4
3.1	L1: In-Memory O(1) LRU Cache	4
3.2	L2: On-Disk WebP Cache	5
4	Concurrent Ingestion Pipeline	5
4.1	Pipeline Topology	5
4.1.1	The Scanner Thread	6
4.1.2	The Worker Pool	6
4.1.3	The Writer Threads	6
4.2	Progress Reporting	6
5	Metadata Indexing	6
5.1	Schema and Index Design	6
5.2	Dynamic Filter Queries	7
6	Frontend Integration	7
7	Future Work / Contributing	8

1 Overview

The Hual backend is split into 4 main pillars:

1. **Thumbnail extraction** - format-specific parsers extract a pre-rendered preview from the RAW file without having to decode any of the sensor's raw pixel data.
2. **Caching** - a two tier system consisting of an in-memory L1 LRU cache (heap-allocated) which stores full resolution previews of recently viewed images to avoid re-parsing the RAW container, and an on-disk L2 cache of downscaled, WebP-compressed grid thumbnails that persists across app restarts.
3. **Concurrent ingestion pipeline** - the scanner thread walks the source directory and feeds raw file bytes to a pool of worker threads, each worker extracts metadata and thumbnails; these are then sent to dedicated (separate) disk writer and database writer threads. All stages are connected by bounded channels to enforce backpressure throughout the pipeline.
4. **Metadata indexing** - image metadata is indexed into an SQLite database via a composite index designed around the filter UI's shape.

To ground these four pillars, it helps to trace what happens to a single photo as it moves through the system end to end:

1. The scanner discovers the file during a directory walk.
2. The file is handed to a worker thread, which picks the right parser based on the file extension.
3. The parser extracts EXIF & thumbnail via zero-copy mmap slicing.
4. The thumbnail is downscaled and cached in L2 as a .webp file and metadata is queued for the database writer.
5. On first UI request, the full preview is parsed once and cached in L1 for subsequent lookups.

2 Thumbnail Extraction

2.1 Zero-Copy mmap Parsing

Irrespective of format, the file is memory-mapped once via `mmap2`, producing a `&u8` view over the file without reading it into a buffer. The OS pages in bytes on demand, so only a handful of specific regions of the byte slice are ever paged in, making the process extremely cheap compared to copying the whole file. Since the input is an arbitrary file from the hard-disk, every read goes through checked slicing (`data.get(range)`), allowing us to produce logged errors from malformed files rather than panic.

2.2 TIFF/EXIF Parser (NEF, ARW, DNG, plain JPEG EXIF)

TIFF-based formats share an 8 byte header. This begins with a two-character byte-order marker - "II" (Intel, little-endian) or "MM" (Motorola, big-endian) - which determines how the subsequent byte fields in the file are interpreted. This is resolved once at the start of parsing which is stored as a property of the `TiffHeader` to avoid re-checking endianness for every field. The rest of the byte header contains a magic number to confirm the file type, and an offset to the first IFD (Image File Directory).

The embedded thumbnail's location is given by tags `0x0201` (offset) and `0x0202` (byte length). IFD0's thumbnail tags are checked before IFD1, as some cameras store higher quality previews

in IFD0, whilst lower resolution thumbnails are found in IFD1. SubIFD is checked last as a fallback option.

Tag `0x8769` in IFD0 holds a `u32` that is interpreted as an absolute byte offset from the start of the file, pointing to the EXIF sub-IFD. The desired EXIF fields are then extracted from standardised tags in this IFD. Rational values (such as f-stop / exposure time) are stored as numerator, denominator pairs until inserted into the database.

2.3 ISO-BMFF Box Parser (Canon CR3)

In 2018, Canon released a new proprietary RAW format `CR3`. Unlike the previous `CR2` format, this is not TIFF based, it is instead based on the ISO-BMFF format, requiring its own, separate parser.

`CR3` is structured as a sequence of self-describing boxes (size, FourCC type, 64-bit extended size (optional), UUID (optional)).

The parser first validates the container by reading the top level boxes to find the `ftyp` box, then verifying its major brand is `"crx "` which confirms the file is a `CR3` as opposed to some other ISO-BMFF-family file format. Among the top level boxes, there is a `uuid` box whose `usertype` matches a specific constant. This is the preview container, which contains an undocumented 8-byte gap which the parser skips, followed by some child boxes. Amongst these is a box named `"PRVW"` (preview). This contains the jpeg thumbnail size at offset 12, with the jpeg bytes themselves at offset 16, which are sliced and copied out using `.to_vec()`.

The EXIF data is located via a separate path through the container. The parser first descends into the top-level `moov` box, and searches its child boxes for a second (distinct) Canon UUID box, inside which a box named `"CMT2"` is found. This box turns out to be a standard TIFF/EXIF blob, so it is simply handed off to the same TIFF parser from [2.2](#).

Since Canon has not officially documented the `CR3` format, locating these boxes and constants is effectively reverse-engineered work. I did not derive these independently, but was able to make use of them thanks to the efforts of others in the open-source community.

2.4 JPEG Parser

In case of a mixture of RAW and JPEG files, Hual supports the import of JPEG files for database indexing and filtering.

JPEG files required a third and distinct parser, since their structure differs from both TIFF and ISO-BMFF. Rather than a directory based layout, JPEG files consist of a flat sequence of segments, each beginning with a two-byte marker (`0xFF` followed by a type identifier byte), typically followed by a two-byte big-endian length for that segment. The parser walks these segments sequentially, looking for an `APP1` (`0xFFE1`) segment whose payload begins with the six bytes `"Exif\0\0"`. The bytes following those six bytes are once again a standard TIFF/EXIF blob, which is again handed off to the same TIFF parser from [2.2](#).

Unlike the RAW formats, a plain JPEG does not require thumbnail extraction since the file itself is already a displayable image, so the thumbnail returned is the original, unchanged input bytes.

2.5 Convergence Design

Thumbnail extraction is dispatched by file extension: `thumbnail.rs` picks the appropriate top level module, each of which uses format specific logic to extract the thumbnail.

EXIF parsing does not diverge in this way. Whilst the format specific modules individually locate the EXIF blob, all three modules then hand the byte range off to a shared TIFF-based parser to extract the desired metadata values.

3 Caching Architecture

3.1 L1: In-Memory O(1) LRU Cache

The L1 cache is a hand-rolled LRU (least recently used) cache, using an array-backed doubly linked list combined with a hashmap, designed to give constant time `get` and `put` operations. It backs the full resolution preview path (`get_preview`), reducing the need for re-parsing RAW containers, as this is an expensive operation.

Due to Rust’s mutable aliasing restrictions for memory safety, the linked list is not implemented traditionally using pointers, instead, an arena-based approach is used.

The system consists of the following core components:

1. **nodes**: `Vec<Node>`: This holds all of the actual data. Instead of allocating memory on the heap for every new node, the cache pre-allocates a fixed-size array. The `prev` and `next` fields in the `Node` struct are `usize` indices pointing to other elements in this array.
2. **map**: `HashMap<K, usize>`: To read or update a key, it must be found in the linked list. To avoid the $O(n)$ traversal of the list, we make use of a hashmap which stores the key and the **index** of that key’s node in the **nodes** array. This allows for $O(1)$ (constant time) access.
3. **free**: `Vec<usize>`: This acts as a stack of the indices in the **nodes** array containing blank nodes. When a new item is inserted, an index is popped from **free**.

The linked list also has two sentinel nodes, **HEAD** and **TAIL**, which are inserted into the **nodes** array at indices 0 and 1 respectively. These ensure that every “real” node has a valid `prev/next` index to link to, which eliminates the need for null-checking at the boundaries of the list.

On a cache hit, the desired node is unlinked from its current position and relinked at the front of the list, to enforce the least recently used ordering. On a miss, a new node is appended at the front of the list, evicting the tail node if no free slots are available.

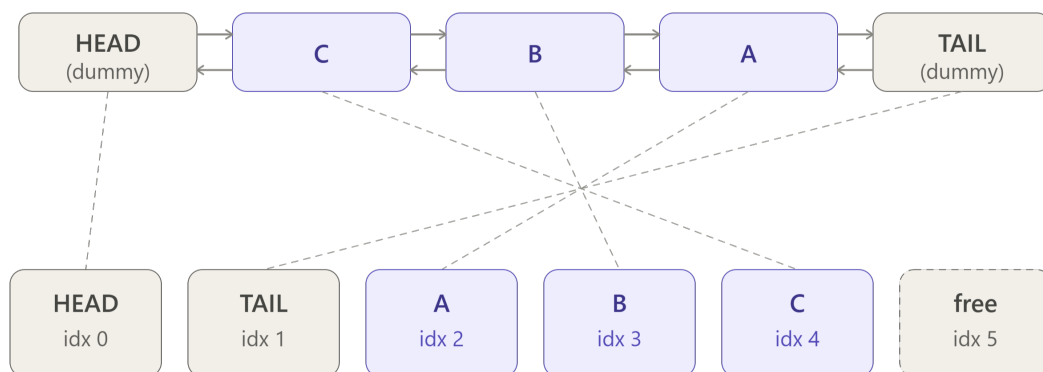


Figure 1: Arena-based doubly linked list

3.2 L2: On-Disk WebP Cache

The L2 cache is an on-disk store designed to persist across app restarts. It caches the small form-factor thumbnail grid previews, rather than the full resolution previews that L1 backs.

Each entry is keyed by an FNV-1a 64-bit hash of the source file's path, which is used as the cache file's name on disk. On a cache miss, the extracted JPEG preview (from section 2) is decoded via the `image` crate, downscaled to a maximum dimension of 256px (if larger), and re-encoded to WebP at quality 80 before being written out. Displayed in the small previews of the photo grid, the drop in visual fidelity is hardly noticeable, and this trade-off allows for a significantly smaller file size.

4 Concurrent Ingestion Pipeline

4.1 Pipeline Topology

The ingestion pipeline is wired together in a `std::thread::scope` call, with the following 4 stages connected via bounded `crossbeam-channels`:

1. a scanner
2. a pool of worker threads
3. a dedicated disk writer
4. a dedicated database writer

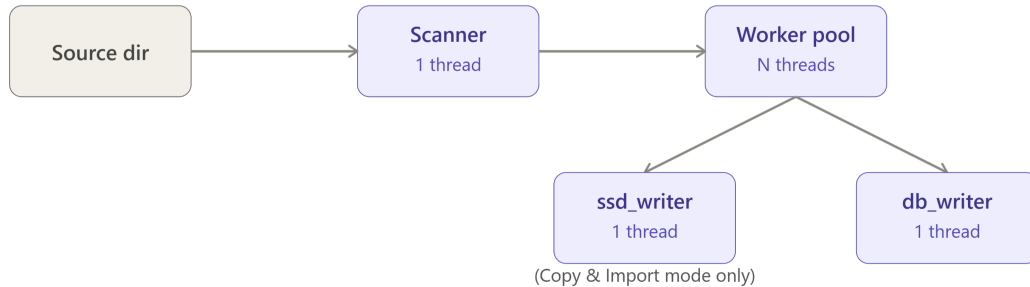


Figure 2: Pipeline topology

Using a scoped thread block, rather than `thread::spawn` gives structured concurrency - the compiler guarantees every spawned thread is joined before the function returns. Hence, there are no detached threads and there is no need for `unsafe`. This approach also means that references needing to be shared across threads do not require an `Arc` but can instead be borrowed directly into worker closures, since the scope guarantees they will outlive all of the threads that borrow them.

As previously mentioned, each stage is connected to the next by a **bounded** channel. This means that a faster stage cannot race ahead of a slower one downstream. If one stage is faster than a future one, the channel will eventually fill to capacity and block future sends, naturally throttling the pipeline to the speed of its slowest stage.

4.1.1 The Scanner Thread

The scanner thread performs a recursive walk of the source directory, descending into child directories whilst avoiding symlinks and the pipeline’s own `.Hual/` directory. This exclusion is important as in “Import Only” mode, the `.Hual/` directory is created inside the folder being scanned, so failure to exclude it would result in the scanner attempting to ingest its own SQ-Lite database and cached thumbnails. If the user has requested a RAW-only import, then JPEG files are filtered out at this stage as well, avoiding unnecessary work being done on them.

4.1.2 The Worker Pool

The worker pool consists of N threads, where N is derived at runtime via `std::thread::available_parallelism()`, which queries the OS for the number of logical CPUs available, allowing the pipeline to scale according to the hardware it is running on.

The threads pull raw file bytes off a shared receiver and independently perform thumbnail and EXIF extraction, as well as L2 cache population. This is the only stage that is parallelized across multiple threads.

4.1.3 The Writer Threads

There are two, separate writer threads: a disk writer and a database writer.

The disk writer thread writes file bytes to their destination path on disk. This is only meaningful in “Copy & Import” mode. In “Import Only” mode, files are not relocated and therefore the worker pool never constructs a write job for this thread to consume.

The database writer is a single thread that inserts one row per file into the SQLite database in order to store the desired metadata values.

The two writers are single-threaded by design, since disk I/O and SQLite writes do not benefit from parallelization in the way that CPU-bound parsing does.

4.2 Progress Reporting

Progress is tracked via a shared `AtomicUsize` counter, incremented once by whichever worker thread finishes processing a file. The use of an atomic here avoids the need for a mutex just to update a single number that is incremented concurrently across threads. Each increment is a single, lock-free hardware operation.

This counter is paired with a callback, `&dyn Fn(usize) + Sync`, passed down from the top-level `run_import` call. The core engine has no dependency on the frontend (Tauri) at all, so it has no knowledge of how progress is displayed. The Tauri layer supplies a concrete callback which throttles UI event emissions via a `Mutex<Instant>` gate, which avoids firing an IPC event after every single file is processed which would be likely to flood the frontend with IPC messages faster than the UI can usefully render updates.

5 Metadata Indexing

5.1 Schema and Index Design

Metadata is stored in SQLite via `rusqlite`’s bundled feature, which compiles SQLite directly into the binary rather than relying on a system-installed library. This removes the need for an external dependency that the user would otherwise need to have installed.

A single table, `photos`, holds one row per imported file:

```
CREATE TABLE IF NOT EXISTS photos (  
  id INTEGER PRIMARY KEY,  
  src_path TEXT NOT NULL UNIQUE,  
  dest_path TEXT NOT NULL,  
  exposure_time REAL,  
  f_stop REAL,  
  focal_length REAL,  
  iso INTEGER  
);  
CREATE INDEX IF NOT EXISTS idx_photos_exif  
ON photos (iso, f_stop, exposure_time, focal_length);
```

`src_path` is marked `UNIQUE`, since it uniquely identifies a source file regardless of import mode. Additionally, the four EXIF fields are nullable, since it is not guaranteed that every file's meta-data contains every field.

The schema currently captures only these four EXIF values, but it is likely that future updates may extend `photos` with additional metadata fields to allow for more filtering options.

A composite index, `idx_photos_exif`, is built over the four (current) EXIF columns - `iso`, `f_stop`, `exposure_time`, `focal_length` - rather than four separate single-column indexes. This is shaped around the UI's query functionality, where users filter by combinations of these fields simultaneously. A composite index lets SQLite satisfy these multi-column range queries using a single index lookup, instead of computing several single-column index results and intersecting them.

5.2 Dynamic Filter Queries

Filtering is implemented as a dynamically constructed, parametrised query, designed to accommodate any combination of the available filters. For each of the numeric EXIF fields, optional minimum and/or maximum bounds contribute clauses in the form `column >= ?N` or `column <= ?N` (as appropriate). All present clauses are joined with `AND` to build the SQL query.

The **rational** EXIF values extracted during parsing (Section 2.2) are converted from raw (`numerator`, `denominator`) pairs to `f64` at insert time. `NULLs` are produced for any field a given file's EXIF data did not contain, and SQLite's `IS NULL` semantics then exclude those rows from any active range filter on that column.

6 Frontend Integration

The frontend is a React + TypeScript application built with Vite, communicating with the Rust backend via five typed Tauri IPC commands (`list_photos`, `get_preview`, `get_webp`, `import_photos`, `pick_dir`). Whilst the frontend is not the focus of this report, there are a few points I felt worth noting.

First, the photo grid uses scroll virtualisation - only the rows of the photo grid currently in view are rendered, with a small overscan buffer above and below used to keep scrolling smooth by pre-rendering a few rows before they come into view. Second, the single-photo Lightbox view's forward and backward navigation reuses the same cache-backed preview-fetch function used to open a photo initially. This is where the L1 cache ties in with the frontend - if a user navigates back to a photo they recently looked at, the fetch call will resolve from the cache, in the same way as if a recently viewed photo was clicked on from the photo grid.

7 Future Work / Contributing

Hual is open source and available on GitHub at <https://github.com/HNarendranath/Hual>, licensed under the MIT license. Whilst the project was largely a learning exercise, and is far from revolutionary, there is still plenty of room left for new features and contributions, spanning frontend improvements, additional metadata fields, and general maintenance. If you're interested in raw byte parsing, systems programming, or just want to poke around some of the cache or thread implementations, issues and pull requests are very welcome.